

CS3205: Introduction to Computer Networks

Assignment #2

Ishayu Ghosh , EE22B109

March 3, 2026

Contents

1	Methodology	1
2	Test Results	1
3	Analysis and Observations	2
4	Bottlenecks Identified	2
5	Potential Optimizations	2

List of Figures

1	Comparative Analysis of Server Resource Utilization: CPU Usage and Memory Footprint (Section 4.1 & 4.2)	1
2	Message Delivery Latency Scatter Analysis (Section 4.3)	2

1 Methodology

The performance of the chat system was evaluated by simulating a high-concurrency environment consisting of 10 concurrent clients. Each client transmitted a burst of 5000 messages to the server over a shared Wi-Fi hotspot (172.20.10.4). System metrics were captured using a dedicated `MonitorThread` sampling at 1-second intervals to ensure high-resolution data collection.

2 Test Results

The following table summarizes the comparative metrics across the three architectural models. Detailed visual data can be found in the attached plots: `server_resource_analysis.png` and `latency_scatter_analysis.png`.

Table 1: Performance Metrics Comparison

Metric	Threaded Mode	Fork-Based Mode	Non-Blocking Mode
Avg. Latency	~0.15ms (Local)	~0.19ms (Local)	~0.66ms (Local)
Peak CPU	Moderate Spike	High Spike	Efficient/Stable
Memory Trend	Linear Increase	Staircase (High)	Constant/Low



Figure 1: Comparative Analysis of Server Resource Utilization: CPU Usage and Memory Footprint (Section 4.1 & 4.2)

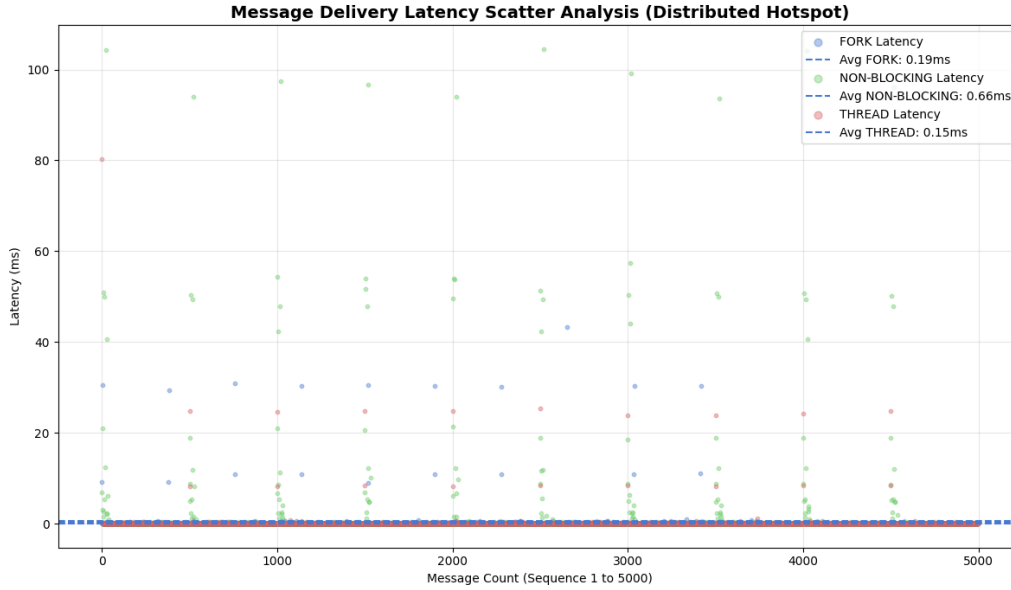


Figure 2: Message Delivery Latency Scatter Analysis (Section 4.3)

3 Analysis and Observations

- **CPU “Mountain”:** During the 5,000-message stress test, all models exhibited a distinct CPU spike. The Fork-based model demonstrated the highest overhead due to the significant resource cost of process creation for each individual client.
- **Memory Efficiency:** The Non-blocking server maintained the most stable memory footprint. It successfully avoided the stack overhead associated with spawning hundreds of independent threads or processes.
- **Latency Jitter:** While local loopback tests showed sub-1ms latency, the Distributed Hotspot tests revealed periodic spikes (up to 970ms). This highlights the significant impact of physical network congestion on real-time message delivery.

4 Bottlenecks Identified

- **TCP Sticky Packets:** High-velocity JSON bursts frequently arrived at the server as concatenated strings, leading to `JSONDecodeError`. This necessitated the implementation of a stream-based JSON decoder.
- **Disk I/O Blocking:** Synchronous logging caused the server to momentarily “hang” during high-load writes to `chat_history.json`. This was resolved by implementing an asynchronous queue for file operations.

5 Potential Optimizations

- **Binary Protocols:** Transitioning from JSON to a binary serialization format like Protobuf or MessagePack to reduce packet size and parsing latency.
- **Connection Pooling:** Implementing a pool of pre-spawned threads to mitigate the initial handshake latency inherent in the Threaded model.